

Session 7

Spring Boot and REST controllers

Spring Boot removes most configuration: an embedded server, auto-configured data source and one property file. REST controllers return JSON instead of views, which is how modern front ends consume the same student data.

Objectives

✕ Do not copy. Read for understanding and the viva

- Complete questions 29 to 32 of the manual: spring boot and rest controllers
- Prepare the deliverable before the lab and finish it during the session
- Be ready to explain every step in the viva

Questions Covered

✕ Do not copy. Read for understanding and the viva

Question	Requirement	Status
Q29	Create a Spring Boot application using Spring Initializer. Add the following...	Complete
Q30	Configure Database settings through the property file in Spring Boot	Complete
Q31	Create JPA Repositories for all entities used in the Student Admission lifecycle	Complete
Q32	Create Rest Controller to fetch Student Information using JPA Repository; the response...	Complete

Preparation

✗ Do not copy. Read for understanding and the viva

- Use Spring Initializr with the listed dependencies; the exact starter names are spring-boot-starter-web, spring-boot-starter-data-jpa, spring-boot-starter-thymeleaf, spring-boot-devtools, spring-boot-starter-actuator and the database driver.
- Database settings go in `application.properties`: url, username, password, `spring.jpa.hibernate.ddl-auto`.
- A `JpaRepository<Student, Long>` interface needs no implementation; Spring generates it.

Sessions 7 to 10 build one project, `admission-api`. Each session adds files to it or replaces files from the session before. Every listing carries its path inside the project as a first-line comment. Nothing in these four sessions was executed here (no JDK, Maven or MySQL on this machine); every listing was checked by reading against the Spring Boot 3.3 and Spring Security 6 APIs.

Question 29

Problem Statement

■ Write in lab record

Create a Spring Boot application using Spring Initializer. Add the following dependencies manually:

1. Spring MVC
2. Hibernate
3. JPA
4. Thymeleaf
5. DevTool
6. Actuator
7. MySQL/MSSQL/Oracle/MongoDB (as per your choice) driver.

Solution

■ Write in lab record

Steps

1. Open start.spring.io in a browser.
2. Fill the left panel: Project `Maven`, Language `Java`, Spring Boot `3.3.4`, Group `in.ignou`, Artifact `admission-api`, Name `admission-api`, Package name `in.ignou.admission`, Packaging `Jar`, Java `17`.
3. Click `Add dependencies` and pick, one by one: `Spring Web`, `Spring Data JPA`, `Thymeleaf`, `Spring Boot DevTools`, `Spring Boot Actuator`, `MySQL Driver`.
4. Click `Generate`; unzip `admission-api.zip` into your workspace.
5. Eclipse: File, Open Projects from File System, Directory, select the `admission-api` folder, Finish. Wait until Maven finishes downloading (bottom-right progress bar).
6. Open `pom.xml` and compare with the listing below. If a dependency is missing, paste its block inside `<dependencies>`, save, then right-click the project, Maven, Update Project.
7. Run once: right-click the project, Run As, Spring Boot App (or `./mvnw spring-boot:run` in a terminal).

Program

- Lab record: every tab is one file of the answer. Write all of them.

pom.xml

pom.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!-- admission-api/pom.xml : generated by Spring Initializr, Spring Boot 3.3
3 <project xmlns="http://maven.apache.org/POM/4.0.0"
4         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven
6 <modelVersion>4.0.0</modelVersion>
7
8 <parent>
9     <groupId>org.springframework.boot</groupId>
10    <artifactId>spring-boot-starter-parent</artifactId>
11    <version>3.3.4</version>
12    <relativePath/>
13 </parent>
14
15 <groupId>in.ignou</groupId>
16 <artifactId>admission-api</artifactId>
17 <version>0.0.1-SNAPSHOT</version>
18 <name>admission-api</name>
19 <description>MCSL-222 Student Admission API (Sessions 7 to 10)</descriptio
20
21 <properties>
22     <java.version>17</java.version>
23 </properties>
24
25 <dependencies>
26     <!-- a. Spring MVC (embedded Tomcat, Jackson for JSON) -->
27     <dependency>
28         <groupId>org.springframework.boot</groupId>
29         <artifactId>spring-boot-starter-web</artifactId>
30     </dependency>
31     <!-- b. Hibernate and c. JPA: one starter brings hibernate-core and Spri
32     <dependency>
33         <groupId>org.springframework.boot</groupId>
34         <artifactId>spring-boot-starter-data-jpa</artifactId>
35     </dependency>
36     <!-- d. Thymeleaf view templates -->
37     <dependency>
38         <groupId>org.springframework.boot</groupId>
39         <artifactId>spring-boot-starter-thymeleaf</artifactId>
40     </dependency>
41     <!-- e. DevTools: automatic restart on save -->
42     <dependency>
43         <groupId>org.springframework.boot</groupId>
```

```
44     <artifactId>spring-boot-devtools</artifactId>
45     <scope>runtime</scope>
46     <optional>true</optional>
47 </dependency>
48 ←!— f. Actuator: /actuator/health, /actuator/metrics ... →
49 <dependency>
50     <groupId>org.springframework.boot</groupId>
51     <artifactId>spring-boot-starter-actuator</artifactId>
52 </dependency>
53 ←!— g. MySQL driver →
54 <dependency>
55     <groupId>com.mysql</groupId>
56     <artifactId>mysql-connector-j</artifactId>
57     <scope>runtime</scope>
58 </dependency>
59 <dependency>
60     <groupId>org.springframework.boot</groupId>
61     <artifactId>spring-boot-starter-test</artifactId>
62     <scope>test</scope>
63 </dependency>
64 </dependencies>
65
66 <build>
67     <plugins>
68         <plugin>
69             <groupId>org.springframework.boot</groupId>
70             <artifactId>spring-boot-maven-plugin</artifactId>
71         </plugin>
72     </plugins>
73 </build>
74 </project>
```

Output

Checked by reading, not executed. The first run stops before Tomcat starts because the data source is not configured yet; that is Question 30:

```
*****  
APPLICATION FAILED TO START  
*****
```

Description:

Failed to configure a DataSource: 'url' attribute is not specified and no embedded datasource could be configured.

Reason: Failed to determine a suitable driver class

After Question 30 the same run ends with lines of this shape:

```
Tomcat initialized with port 8080 (http)  
HikariPool-1 - Start completed.  
Tomcat started on port 8080 (http) with context path '/'  
Started AdmissionApiApplication in 4.1 seconds (process running for 4.6)
```

Explanation

The seven items in the question map to six Maven artifacts. Spring MVC is `spring-boot-starter-web` (it also brings embedded Tomcat and Jackson). Hibernate and JPA arrive together in `spring-boot-starter-data-jpa`; Hibernate is the JPA implementation, so there is no separate starter. Thymeleaf, DevTools and Actuator are their own starters. The driver is `mysql-connector-j` with `runtime` scope because our code never imports a MySQL class; only JDBC needs it at run time. `spring-boot-starter-parent` fixes every version, so no `<version>` tag appears under the dependencies. `@SpringBootApplication` turns on auto-configuration and scans `in.ignou.admission` and its sub-packages for `@Entity`, `@Repository`, `@Service` and `@Controller` classes.

Question 30

Problem Statement

■ Write in lab record

Configure Database settings through the property file in Spring Boot.

Solution

■ Write in lab record

Steps

1. Start MySQL and create the schema: `mysql -u root -p` then `CREATE DATABASE admission_db;`. (The `createDatabaseIfNotExist=true` flag in the URL does the same job if the account may create schemas.)
2. Open `src/main/resources/application.properties` (empty after Initializr) and paste the listing.
3. Change `spring.datasource.username` and `password` to your MySQL account.
4. Run the application; watch the console for `HikariPool-1 - Start completed`.

Configuration

application.properties

```
1 # src/main/resources/application.properties (Session 7, Q30)
2 spring.application.name=admission-api
3 server.port=8080
4
5 # --- MySQL connection ---
6 # create the schema first: CREATE DATABASE admission_db;
7 spring.datasource.url=jdbc:mysql://localhost:3306/admission_db?createDatabaseIfNotExist=true
8 spring.datasource.username=root
9 spring.datasource.password=root
10 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
11
12 # --- JPA / Hibernate ---
13 # update = create missing tables and columns from the @Entity classes, never
14 spring.jpa.hibernate.ddl-auto=update
15 spring.jpa.show-sql=true
16 spring.jpa.properties.hibernate.format_sql=true
17
18 # --- seed data (src/main/resources/data.sql) ---
19 # run data.sql after Hibernate has created the tables, on a real (non-embedd
20 spring.jpa.defer-datasource-initialization=true
21 spring.sql.init.mode=always
```

Output

Checked by reading. Expected console lines on a successful start:

```
HikariPool-1 - Starting...
HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@5c3b1f2a
HikariPool-1 - Start completed.
HHH000412: Hibernate ORM core version 6.5.3.Final
HHH10001005: Database info:
    Database JDBC URL [jdbc:mysql://localhost:3306/admission_db]
    Database driver: com.mysql.cj.jdbc.Driver
    Database version: 8.0.36
```

A wrong password fails at the pool, not at the query:

```
HikariPool-1 - Exception during pool initialization.
java.sql.SQLException: Access denied for user 'root'@'localhost' (using password: YES)
```

Explanation

Property	Effect
<code>spring.datasource.url</code>	JDBC URL; host, port, schema and flags. <code>serverTimezone</code> stops the "server time zone value is unrecognized" error.
<code>spring.datasource.username</code> , <code>password</code>	MySQL account. Boot creates a HikariCP pool from these.
<code>spring.datasource.driver-class-name</code>	Optional; Boot infers it from the URL. Kept so the driver name is visible in the record.
<code>spring.jpa.hibernate.ddl-auto=update</code>	Hibernate compares entities with tables at start and issues <code>CREATE TABLE</code> or <code>ALTER TABLE ADD</code> as needed. It never drops columns. Use <code>validate</code> in production.
<code>spring.jpa.show-sql=true</code>	Prints every SQL statement; useful evidence for the lab record.
<code>spring.jpa.defer-datasource-initialization=true</code>	Runs <code>data.sql</code> after Hibernate has created the tables. Without it the inserts run first and fail.
<code>spring.sql.init.mode=always</code>	Boot runs <code>data.sql</code> only for embedded databases by default; <code>always</code> enables it for MySQL.

Question 31

Problem Statement

■ Write in lab record

Create JPA Repositories for all entities used in the Student Admission lifecycle.

Solution

■ Write in lab record

Steps

1. Create the package `in.ignou.admission.entity` and add `Student`, `Programme`, `Course`, `Admission` and the enum `AdmissionStatus`.
2. Create the package `in.ignou.admission.repository` and add one interface per entity extending `JpaRepository<Entity, Long>`.
3. Run the application. With `ddl-auto=update` and `show-sql=true` the console prints the `create table` statements the first time.
4. Check in MySQL: `USE admission_db; SHOW TABLES;` lists `admission`, `course`, `programme`, `student`.

Program

The life cycle has four entities. A Student applies to a Programme; the application is an Admission whose status moves APPLIED, VERIFIED, APPROVED or REJECTED; a Programme owns Courses.

■ Lab record: every tab is one file of the answer. Write all of them.

Student.java

Student.java

```
1 // src/main/java/in/ignou/admission/entity/Student.java
2 package in.ignou.admission.entity;
3
4 import java.time.LocalDate;
5
6 import jakarta.persistence.Column;
7 import jakarta.persistence.Entity;
8 import jakarta.persistence.GeneratedValue;
9 import jakarta.persistence.GenerationType;
10 import jakarta.persistence.Id;
11
12 /** Applicant. Maps to table `student` (Hibernate default: snake_case of the
13 @Entity
14 public class Student {
15
16     @Id
17     @GeneratedValue(strategy = GenerationType.IDENTITY)
18     private Long id;
19
20     @Column(nullable = false, length = 80)
21     private String name;
22
23     @Column(nullable = false, unique = true, length = 120)
24     private String email;
25
26     @Column(length = 15)
27     private String phone;
28
29     @Column(length = 60)
30     private String city;
31
32     private LocalDate dateOfBirth;
33
34     public Long getId() { return id; }
35     public void setId(Long id) { this.id = id; }
36     public String getName() { return name; }
37     public void setName(String name) { this.name = name; }
38     public String getEmail() { return email; }
39     public void setEmail(String email) { this.email = email; }
40     public String getPhone() { return phone; }
41     public void setPhone(String phone) { this.phone = phone; }
42     public String getCity() { return city; }
43     public void setCity(String city) { this.city = city; }
```

```
44     public LocalDate getDateOfBirth() { return dateOfBirth; }
45     public void setDateOfBirth(LocalDate dateOfBirth) { this.dateOfBirth = d
46 }
```

One repository per entity:

- Lab record: every tab is one file of the answer. Write all of them.

StudentRepository.java

StudentRepository.java

```
1 // src/main/java/in/ignou/admission/repository/StudentRepository.java
2 package in.ignou.admission.repository;
3
4 import java.util.List;
5 import java.util.Optional;
6
7 import org.springframework.data.jpa.repository.JpaRepository;
8
9 import in.ignou.admission.entity.Student;
10
11 /** No implementation needed: Spring Data generates it at start-up from the
12 public interface StudentRepository extends JpaRepository<Student, Long> {
13
14     Optional<Student> findByEmail(String email);
15
16     List<Student> findByCityIgnoreCase(String city);
17 }
```

Output

Checked by reading. Hibernate 6.5 with the MySQL dialect generates DDL of this shape on the first start (order may differ):

SQL

```
1 create table programme (  
2     duration_years integer not null,  
3     id bigint not null auto_increment,  
4     code varchar(10) not null,  
5     name varchar(100) not null,  
6     primary key (id)  
7 ) engine=InnoDB;  
8 alter table programme add constraint UK_programme_code unique (code);  
9  
10 create table student (  
11     date_of_birth date,  
12     id bigint not null auto_increment,  
13     phone varchar(15),  
14     city varchar(60),  
15     name varchar(80) not null,  
16     email varchar(120) not null,  
17     primary key (id)  
18 ) engine=InnoDB;  
19 alter table student add constraint UK_student_email unique (email);  
20  
21 create table course (  
22     credits integer not null,  
23     id bigint not null auto_increment,  
24     programme_id bigint not null,  
25     code varchar(10) not null,  
26     title varchar(120) not null,  
27     primary key (id)  
28 ) engine=InnoDB;  
29 alter table course add constraint FK_course_programme foreign key (programme  
30  
31 create table admission (  
32     applied_on date,  
33     id bigint not null auto_increment,  
34     programme_id bigint not null,  
35     student_id bigint not null,  
36     status enum ('APPLIED', 'VERIFIED', 'APPROVED', 'REJECTED') not null,  
37     primary key (id)  
38 ) engine=InnoDB;
```

And the start-up log names the repositories it built:

```
Bootstrapping Spring Data JPA repositories in DEFAULT mode.  
Finished Spring Data repository scanning in 41 ms. Found 4 JPA repository interfaces.
```

Explanation

`JpaRepository<T, ID>` already declares `save`, `findById`, `findAll`, `existsById`, `count`, `deleteById` and paging variants. Spring Data creates a proxy class for each interface at start-up, so no implementation is written. Extra methods are derived from their names:

`findByCityIgnoreCase(String)` becomes `select s from Student s where upper(s.city) = upper(?)`; `findByProgrammeCode(String)` walks the `programme` association and compares `programme.code`. A misspelt property name fails at start-up with `No property 'citty' found for type 'Student'`, which is the check that the name is right.

Hibernate default naming converts `dateOfBirth` to `date_of_birth` and the class name `Student` to table `student`. The `@ManyToOne` fields become foreign-key columns named by `@JoinColumn`. `@Enumerated(EnumType.STRING)` stores `APPROVED` as text; without it Hibernate stores the ordinal 2, which breaks the moment someone reorders the enum.

Question 32

Problem Statement

■ Write in lab record

Create Rest Controller to fetch Student Information using JPA Repository; the response should display in JSON format.

Solution

■ Write in lab record

Steps

1. Create the package `in.ignou.admission.web` and add `StudentRestController`.
2. Add `data.sql` under `src/main/resources` so the table has rows to fetch (the properties from Question 30 already enable it).
3. Restart the application. Handler mappings are logged only at DEBUG level, so the request test below is the real check.

4. Run the curl commands in a terminal, or open `http://localhost:8080/api/students` in a browser.

Program

- Lab record: every tab is one file of the answer. Write all of them.

StudentRestController.java

StudentRestController.java

```
1 // src/main/java/in/ignou/admission/web/StudentRestController.java (Session 07)
2 package in.ignou.admission.web;
3
4 import java.util.List;
5
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.GetMapping;
8 import org.springframework.web.bind.annotation.PathVariable;
9 import org.springframework.web.bind.annotation.RequestMapping;
10 import org.springframework.web.bind.annotation.RequestParam;
11 import org.springframework.web.bind.annotation.RestController;
12
13 import in.ignou.admission.entity.Student;
14 import in.ignou.admission.repository.StudentRepository;
15
16 /**
17  * @RestController = @Controller + @ResponseBody: every return value is written
18  * to the HTTP body as JSON by Jackson instead of being resolved as a view name
19  */
20 @RestController
21 @RequestMapping("/api/students")
22 public class StudentRestController {
23
24     private final StudentRepository students;
25
26     // one constructor: Spring injects the repository, no @Autowired needed
27     public StudentRestController(StudentRepository students) {
28         this.students = students;
29     }
30
31     /** GET /api/students or GET /api/students?city=Jaipur */
32     @GetMapping
33     public List<Student> all(@RequestParam(required = false) String city) {
34         return city == null ? students.findAll() : students.findByCityIgnoreCase(city);
35     }
36
37     /** GET /api/students/{id} → 200 with the student, or 404 with an empty body */
38     @GetMapping("/{id}")
39     public ResponseEntity<Student> one(@PathVariable Long id) {
40         return students.findById(id)
41             .map(ResponseEntity::ok)
42             .orElse(ResponseEntity.notFound().build());
43     }
44 }
```

```
43     }  
44 }
```

Output

Checked by reading. Every endpoint with its curl command and expected response:

```
1 curl -s http://localhost:8080/api/students
```

BASH

```
1 [{"id":1,"name":"Asha Verma","email":"asha@example.com","phone":"9876543210"}  
2  {"id":2,"name":"Ravi Kumar","email":"ravi@example.com","phone":"9123456780"}]
```

```
1 curl -s http://localhost:8080/api/students/1
```

BASH

```
{"id":1,"name":"Asha Verma","email":"asha@example.com","phone":"9876543210","city":"Jaipur","dateOfBirth":"2002-03-14"}
```

```
1 curl -s "http://localhost:8080/api/students?city=patna"
```

BASH

```
[{"id":2,"name":"Ravi Kumar","email":"ravi@example.com","phone":"9123456780","city":"Patna","dateOfBirth":"2001-11-02"}]
```

```
1 curl -i http://localhost:8080/api/students/99
```

BASH

```
HTTP/1.1 404  
Content-Length: 0
```

The console shows the SQL Hibernate ran for the first call:

```
Hibernate: select s1_0.id,s1_0.city,s1_0.date_of_birth,s1_0.email,s1_0.name,s1_0.phone from student s1_0
```

Explanation

`@RestController` is `@Controller` plus `@ResponseBody`: the returned `List<Student>` is not a view name, it is handed to Jackson, which walks the getters and writes JSON with `Content-Type: application/json`. `LocalDate` is written as `2002-03-14` because Boot registers the

`JavaTimeModule` and turns off timestamp output. The constructor takes `StudentRepository`; with one constructor Spring injects it without `@Autowired`. `ResponseEntity` is used only where the status code varies: `findById` returns an `Optional`, `map(ResponseEntity::ok)` gives 200 with a body, `orElse(notFound())` gives 404 with none. `@RequestParam(required = false)` lets the same method serve the full list and the filtered list.

Viva Questions

× Do not copy. Read for understanding and the viva

- **Q:** What does `@SpringBootApplication` combine? **A:** `@Configuration`, `@EnableAutoConfiguration` and `@ComponentScan` on the package of the class.
- **Q:** Why is there no `<version>` under each dependency? **A:** `spring-boot-starter-parent` manages versions through its `dependencyManagement` section, so all starters agree.
- **Q:** What does `ddl-auto=update` do and why not use it in production? **A:** It adds missing tables and columns from the entities at start-up. It never drops or renames, so the schema drifts; production uses `validate` with migration scripts.
- **Q:** How does Spring Data implement `findByCityIgnoreCase` without code? **A:** It parses the method name into a JPQL query at start-up and generates a proxy that runs it.
- **Q:** What is the difference between `@Controller` and `@RestController`? **A:** `@RestController` adds `@ResponseBody`, so return values are written to the response as JSON instead of resolved as view names.
- **Q:** Why does `data.sql` need `defer-datasource-initialization`? **A:** Boot runs SQL scripts before JPA starts by default; deferring runs them after Hibernate created the tables.
- **Q:** Why is `mysql-connector-j` scoped `runtime`? **A:** Our code compiles against JDBC interfaces only; the driver is needed only when the application runs.
- **Q:** What does `Optional.map(ResponseEntity::ok).orElse(notFound().build())` return for a missing id? **A:** A `ResponseEntity` with status 404 and an empty body.

Common Mistakes

× Do not copy. Read for understanding and the viva

- Choosing Spring Boot 2.x on Initializr and then writing `jakarta.persistence` imports; 2.x uses `javax.persistence` and nothing compiles.

- Placing entity or controller packages outside `in.ignou.admission`; component scan never finds them and the endpoint returns 404 with no error in the log.
- Forgetting `spring.sql.init.mode=always`, so `data.sql` is silently skipped on MySQL and every GET returns `[]`.
- Using `@Enumerated` without `EnumType.STRING`, storing ordinals that break when the enum changes.
- Adding a getter-less field to an entity and wondering why it is missing from the JSON; Jackson serialises getters.
- Running before MySQL is up. The pool fails with `Communications link failure`, which students misread as a code error.

Session Summary

■ Write in lab record

- Initializr settings and the `pom.xml` with the six starters plus the MySQL driver
- `application.properties` with the datasource, `ddl-auto=update`, `show-sql` and the `data.sql` switches
- Entity classes `Student`, `Programme`, `Course`, `Admission` and the `AdmissionStatus` enum, with the generated `CREATE TABLE` statements
- The four `JpaRepository` interfaces with their derived query methods
- `StudentRestController` with the four curl calls and their JSON responses, including the 404 case

 [Edit this page](#)

[< Previous](#)
Session 6

Next >
[Session 8](#)